Tool Design & Orchestration

AKD Workshop

Leo · 10:30–11:30

01 Tool fundamentals

02 Round-trip mechanics

03 Descriptions & schemas

04 Pydantic + BaseTool

05 Error handling & hints

06 MCP architecture

07 CARE integration

The Mental Model

A tool is a function signature the model reads as documentation

The Agentic Loop

1. **Model reads tool definitions**
   (from context)
2. **Model emits a tool_use block**
   (name + arguments)
3. **Your code runs the tool**
4. **You return a tool_result**
   (to the model)
5. **Model continues reasoning**
   (with that result)

Everything the model knows about your tool is text.

There is no magic —
just well-written descriptions.

The model never executes your tool. It reads its name and description, decides whether to call it, and emits a structured request. You execute it.

What the Model Actually Sees

Tools are injected into the context window at call time

```
client.messages.create(
    model="claude-opus-4-...",
    max_tokens=1024,
    tools=[
        {
            "name": "search_datasets",
            "description":
                "Search NASA CMR catalog...",
            "input_schema": {
                "type": "object",
                "properties": {
                    "keyword": {
                        "type": "string",
                        "description":
                          "Scientific keyword..."
                    }
                },
                "required": ["keyword"]
            }
        }
    ],
    messages=[{
        "role": "user",
        "content": "Find SST datasets"
    }]
)
```

Key takeaways

tools= is just a parameter

A list of JSON objects, not a registry or special config file.

Injected at call time

Definitions live in your codebase, nowhere else.

Text all the way down

Name, description, and schema are all documentation.

No hidden magic

Whatever you pass is exactly what the model sees.

The Tool Call Round-Trip

What the response looks like — and how you handle it

① Model responds

```
# response.content:
[{
  "type": "tool_use",
  "id": "toolu_01XFy...",
  "name": "search_datasets",
  "input": {
    "keyword":
      "sea surface temp"
  }
}]
```

② You run the tool

```
# Execute the function
result = run_my_search(
  keyword=
    "sea surface temp"
)

# Returns your data:
# {
#   datasets: [...],
#   count: 42
# }
```

③ Send result back

```
# Append to messages:
{
  "role": "assistant",
  "content":
    response.content
},
{
  "role": "user",
  "content": [{
    "type": "tool_result",
    "tool_use_id":
      "toolu_01XFy...",
    "content": str(result)
  }]
}
```

The round-trip is explicit in your code. You control what the model gets back. This is where error handling, retries, and result shaping live.

Tool Descriptions Are Context Budget

Every word costs tokens — and shapes model behavior

❌ BLOATED

```
{
  "name": "search",
  "description": """This tool can
  be used to search for many
  different types of information.
  It can search datasets, papers,
  concepts, terminology, missions,
  instruments, and more. Use it
  when you want to find something.
  It connects to NASA CMR, STI
  portal, and ADS..."""
}
```

→

✅ PRECISE

```
{
  "name": "search_cmr_datasets",
  "description": (
    "Search the NASA CMR catalog"
    " for Earth science datasets"
    " by scientific keyword,"
    " instrument, or mission name."
    " Returns dataset concept IDs"
    " and metadata."
    " Use for data discovery only"
    " — NOT for papers or docs."
  )
}
```

System prompt + conversation history + tool definitions all compete for the same context window. Bloated tools degrade performance.

Rule of thumb: *If you can't describe the tool in two sentences, it's probably doing too many things.*

The Three Rules of Good Tool Descriptions

Name · Description · Schema — each does different work

1 Name

Unambiguous verb-noun

```
"search"
"get_data"
"fetch"
```

```
"search_cmr_datasets"
"get_dataset_granules"
"fetch_sti_paper_metadata"
```

Unambiguous even alongside 10 other tools

2 Description

Tell it when NOT to use

```
"Search for Earth
science datasets."
```

```
"Search NASA CMR for datasets.
Use for data discovery.
NOT for papers or doc search."
```

Exclusion conditions prevent overlap errors

3 Schema fields

Docstrings for the model

```
"keyword": {"type": "string"}
"start_date": {"type": "string"}
"limit": {"type": "integer"}
```

```
"keyword": {type: string,
  desc: "MODIS, sea ice..."}
"start_date": {"ISO 8601,
  omit for all periods"}
"limit": {default:10, max:50}
```

Annotated — model knows how to populate it

The one most people skip: What should the model NOT use it for?

Pydantic for Tool Schemas

Stop hand-writing JSON — let your types do the work

```python
from pydantic import BaseModel, Field
from typing import Optional

class SearchDatasetsInput(BaseModel):
    keyword: str = Field(
        description="Scientific keyword or mission"
                    " (e.g. 'MODIS', 'sea ice extent')"
    )
    start_date: Optional[str] = Field(
        default=None,
        description="ISO 8601 (YYYY-MM-DD). Omit for all time."
    )
    limit: int = Field(
        default=10, ge=1, le=50,
        description="Max results. Default 10, max 50."
    )

class SearchDatasetsOutput(BaseModel):
    status: str    # "success" | "no_results" | "rate_limited" | "error"
    message: str   # agent hint — what to do next
    results: list[dict]
    retry: bool = False

# Schema is generated — always in sync with your code
tool_schema = SearchDatasetsInput.model_json_schema()
tool = {"name": "...", "input_schema": tool_schema}
```

Where this pays off

Model-readable Field descriptions

Field() descriptions become what the model reads — written once, never out of sync.

Validation before execution

Malformed tool calls surface as clean ValidationError, not cryptic runtime crashes.

Structured output contract

Output model guarantees status, message, retry are always present for agent hints.

CARE auditability

Input/output contracts are explicit, diffable, and version-controlled — exactly what CARE requires.

The AKD BaseTool Pattern

The tool class is the schema, the description, and the implementation

```python
from akd._base import InputSchema, OutputSchema
from akd.tools import BaseTool, BaseToolConfig
from akd_ext.mcp import mcp_tool
from pydantic import Field

class SearchDatasetsInput(InputSchema):
    keyword: str = Field(description="Scientific keyword...")
    limit: int = Field(default=10, ge=1, le=50, ...)

class SearchDatasetsOutput(OutputSchema):
    status: str
    message: str
    results: list[dict]

@mcp_tool   # marks class, registers in MCPToolRegistry
class SearchDatasetsTool(
    BaseTool[SearchDatasetsInput, SearchDatasetsOutput]):
    """Search the NASA CMR catalog for Earth science datasets
    by keyword, instrument, or mission.
    Use for data discovery only — NOT for papers or docs."""

    input_schema = SearchDatasetsInput
    output_schema = SearchDatasetsOutput

    async def _arun(self, p: SearchDatasetsInput
                    ) -> SearchDatasetsOutput:
        results = await cmr_api.search(keyword=p.keyword)
        return SearchDatasetsOutput(
            status="success", message="", results=results)
```

`@mcp_tool`

Sets cls._is_mcp_tool = True.
Registers in MCPToolRegistry.
Registration is class-level;
configuration at instantiation.

`BaseTool`

as_function() ⟶ FastMCP async fn
as_tool_definition() ⟶ Anthropic SDK dict
Description from class docstring.

`Key insight`

You never hand-write a JSON schema dict.
InputSchema / OutputSchema own the contract.
BaseTool owns name, description, wiring.

Registering and Wiring AKD Tools

From class definition to running MCP server — three steps

1 `tools/__init__.py`

```
from akd.tools import BaseToolConfig
from .geocode.geocode_location import (
    GeoCodeLocationTool)
from .datasets.query_active_fires import (
    QueryActiveFiresTool)

# Tool names must match CARE artifact
TOOLS = [
    GeoCodeLocationTool(config=
        BaseToolConfig(
            name="geocode_location")),
    QueryActiveFiresTool(config=
        BaseToolConfig(
            name="query_active_fires")),
    # add new tools here
]
```

2 `mcp_server.py`

```
from fastmcp import FastMCP
from akd_ext.mcp.converter import (
    tool_converter,
    register_mcp_tool)
from tools import TOOLS

mcp = FastMCP("geoai-tutorial-tools")

for tool in TOOLS:
    # tool_converter: calls
    #   tool.as_function(mode="python")
    # register_mcp_tool: calls
    #   mcp.tool(name=...)(fn)
    register_mcp_tool(
        tool_converter(tool), mcp)

mcp.run(transport="sse",
    host="127.0.0.1", port=8080)
```

3 `CARE artifact`

```
# tools/geocode_location.md

## Tool: geocode_location

Description: Convert a place name
or address to lat/lon coordinates.

Input:
  location_name (string, required)

Output:
  lat, lon, formatted_address,
  status, message

Do NOT use for: reverse geocoding,
batch operations, or address
validation.
```

Adding a new tool is exactly three changes: write the class file, add it to TOOLS, document it in the CARE artifact. The server picks it up on next restart.

Error Handling and Agent Hints

The tool result is your last line of communication with the model

```
def search_cmr_datasets(keyword, start_date=None, limit=10):
    try:
        results = cmr_api.search(keyword=keyword, ...)

        if not results:
            return {
                "status": "no_results",
                "message": f"No datasets for '{keyword}'. "
                           "Broaden keyword or remove date filter.",
                "results": []
            }
        return {"status": "success", "results": results}

    except CMRRateLimitError:
        return {
            "status": "rate_limited",
            "message": "Rate limit hit. Retry after a short wait.",
            "retry": True
        }

    except CMRConnectionError as e:
        return {
            "status": "error",
            "message": f"CMR unavailable: {e}. Do not retry.",
            "retry": False
```

Status values & model behavior

`success`

Proceed normally

`no_results`

Broaden query or pivot strategy

`rate_limited`

Wait and retry the call

`error`

Inform user, do not retry

status, message, and retry are not for your logs — they are instructions to the model about what to do next.

What MCP Is (and Why It Exists)

The problem: tools defined ad hoc per agent don't compose

Without MCP

- Duplicated logic across agents
- Inconsistent descriptions for same API
- No shared testing or versioning
- Tool definitions die with one agent

VS

With MCP

- 1 server definition → N agent clients
- Consistent descriptions, reviewed once
- Discovery + versioning built in
- Adding a tool updates all agents

MCP (Model Context Protocol) — A server exposes tools via a standardized protocol. Any compatible client can connect, discover available tools, and call them without knowing the underlying implementation.

MCP in Code — Server & Client

Define once, expose to any client · Any agent discovers and calls tools the same way

Server side — AKD ext pattern

```
# mcp_server.py — FastMCP + akd_ext
from fastmcp import FastMCP
from akd_ext.mcp.converter import (
    tool_converter, register_mcp_tool)
from tools import TOOLS  # all BaseTool instances

mcp = FastMCP(
    "geoai-tutorial-tools",
    instructions="Geospatial tools...",
)

for tool in TOOLS:
    # tool_converter → tool.as_function()
    #   typed async fn from input_schema
    # register_mcp_tool → mcp.tool()(fn)
    register_mcp_tool(
        tool_converter(tool), mcp)

mcp.run(transport="sse",
        host="127.0.0.1", port=8080)
```

Client side — agent discovers tools at runtime

```
# agent.py
async with ClientSession(read, write) as session:

    # Discover tools at runtime
    await session.initialize()
    mcp_tools = await session.list_tools()

    # Convert to Anthropic format
    tools = [
        {
            "name": t.name,
            "description": t.description,
            "input_schema": t.inputSchema
        }
        for t in mcp_tools.tools
    ]

    response = client.messages.create(
        model="claude-opus-4-...",
        tools=tools,   # ← discovered, not hardcoded
        messages=[{"role": "user",
                   "content": user_query}]
    )
```

The agent has no hardcoded knowledge of what tools exist. It asks the server. This is what makes tool sets composable and maintainable across agents.

MCP and CARE — Why They Belong Together

A tool without a CARE artifact is a shadow tool

CARE Target	Consequence of Registering Without a CARE Artifact
Tool Orchestration	No approved description, naming, or error policy — model behavior is undefined
Reasoning Policy	Phase 3 guardrails were written for a specific tool set — new tools have none
Evaluation	Benchmark doesn't cover it — failures are invisible to everyone
Auditability	No version history — regressions can't be traced to a specific change
SME Authority	Developer made a unilateral domain call — bypassing the people who know the science

`CARE artifact`	→	`MCP server`	→	`@mcp_tool class`
The tool's social contract		Where that contract lives		Where it's enforced

The CARE artifact is the tool's social contract. The MCP server is where that contract lives. The @mcp_tool class is where it's implemented and enforced.